CMP1020 - Học sâu

BÀI 5: MẠNG NEURON ĐỆ QUY (RNN)

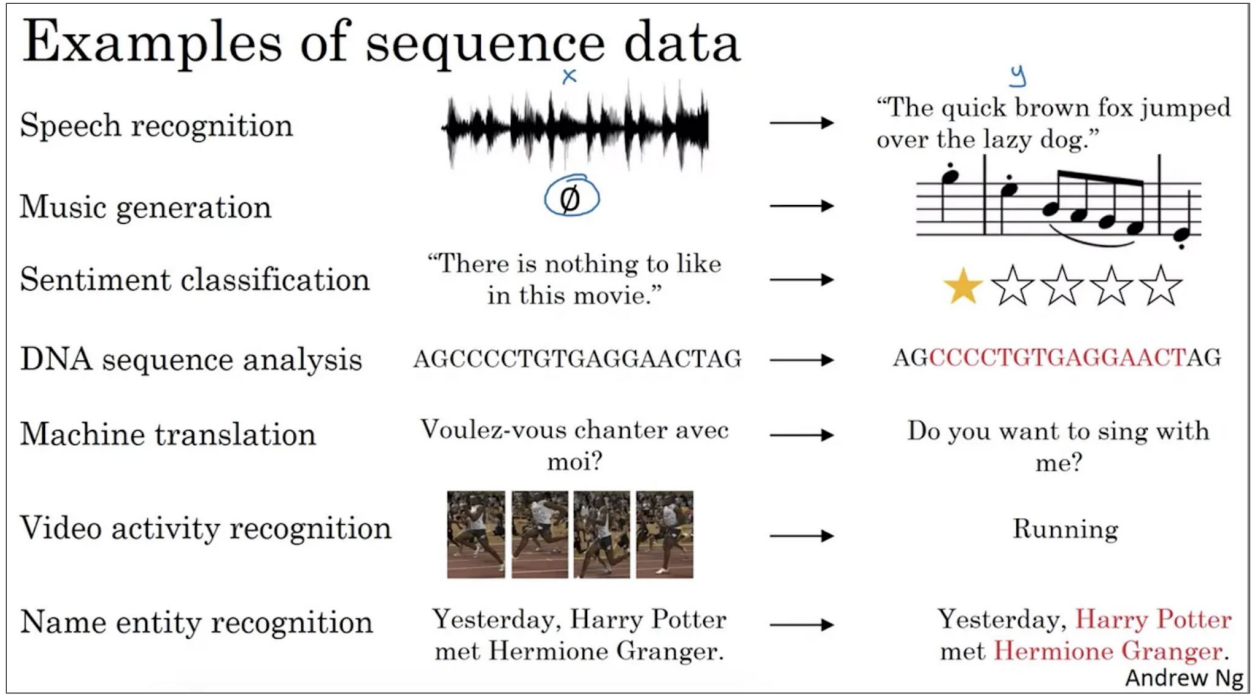
Giảng viên: TS. Bùi Danh Hường

MỤC TIÊU BÀI HỌC

- Hiểu đặc điểm của dữ liệu tuần tự (Sequence Data).
- Nắm vững kiến trúc và công thức toán học của RNN.
- Hiểu cơ chế BPTT (Backpropagation Through Time) và vấn đề biến mất đạo hàm.
- Nắm vững kiến trúc LSTM (Long Short-Term Memory).
- Ứng dụng: Mô tả hình ảnh và Phân tích cảm xúc.

DỮ LIỆU TUẦN TỰ LÀ GÌ?

- **Định nghĩa:** Là dữ liệu mà thứ tự các phần tử mang thông tin quan trọng. Nếu đảo lộn, ý nghĩa sẽ thay đổi hoặc mất đi.
- **Ví dụ:**
  - **Văn bản:** "Tôi yêu Deep Learning" (Khác "Deep Learning yêu Tôi").
  - **Âm thanh:** Chuỗi sóng âm theo thời gian.
  - **Chuỗi thời gian:** Giá chứng khoán, nhiệt độ.
  - **Video:** Chuỗi khung hình liên tiếp.



HẠN CHẾ CỦA MẠNG NEURON TRUYỀN THỐNG

Đầu vào cố định

Yêu cầu input size cố định (vd ảnh 28x28). Không linh hoạt với câu dài/ngắn khác nhau.

Độc lập

Giả định các mẫu dữ liệu độc lập nhau. Không phù hợp với dữ liệu chuỗi.

Không bộ nhớ

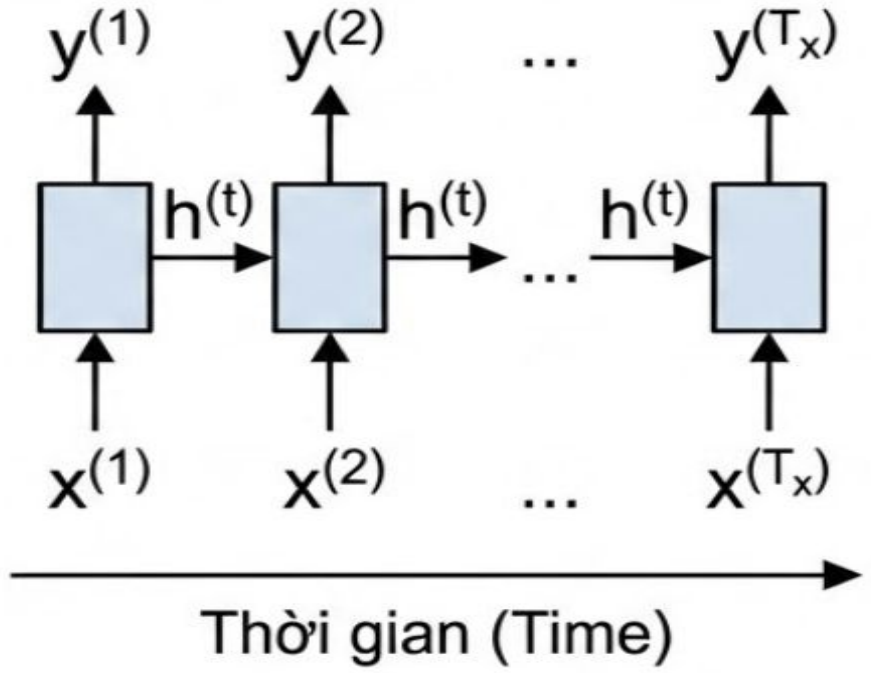
Không nhớ được thông tin từ quá khứ để xử lý hiện tại (Vd: Cần biết "Học" để đoán "Sâu").

MÔ HÌNH TUẦN TỰ (SEQUENCE MODELS)

Ra đời để xử lý dữ liệu có độ dài T_x thay đổi.

Ý tưởng: Xử lý từng phần tử của chuỗi lần lượt, đồng thời giữ lại "trạng thái" (bộ nhớ) về những gì đã xem qua.

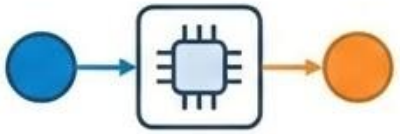
Ký hiệu chuỗi đầu vào: $x^{(1)}, x^{(2)}, \dots, x^{(t)}, \dots, x^{(T_x)}$.



PHÂN LOẠI KIẾN TRÚC RNN

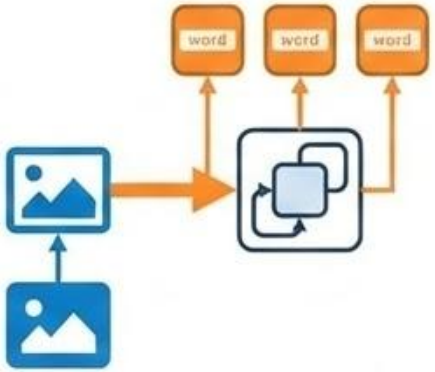
One-to-One

Mạng truyền thống (Không phải RNN).



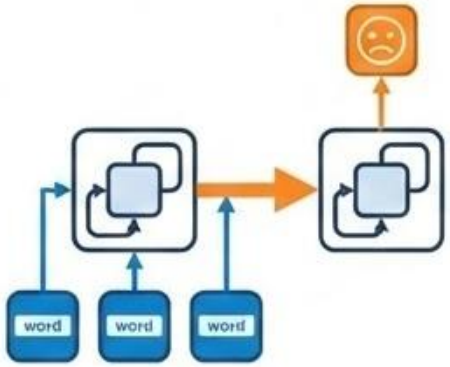
One-to-Many

1 Input → Chuỗi Output
VD: Mô tả ảnh - Image Captioning.



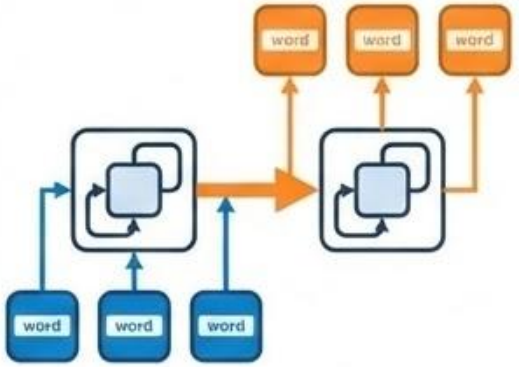
Many-to-One

Chuỗi Input → 1 Output
VD: Phân loại cảm xúc.



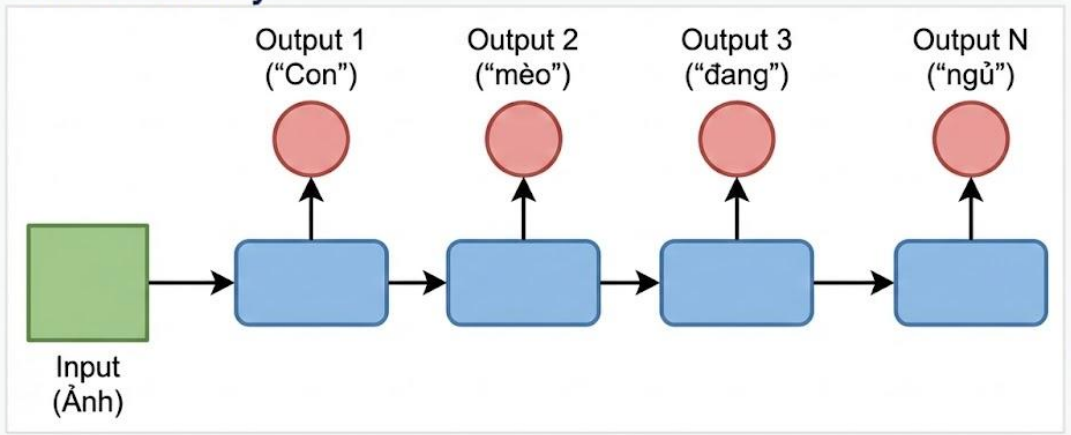
Many-to-Many

Chuỗi Input → Chuỗi Output
VD: Dịch máy, Chatbot.



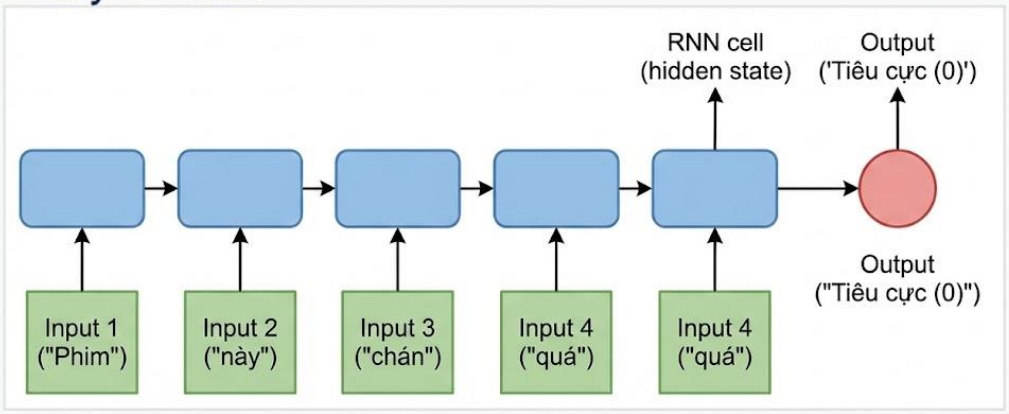
VÍ DỤ MINH HỌA

One-to-Many



Input: Ánh → Output: "Con mèo đang ngủ".

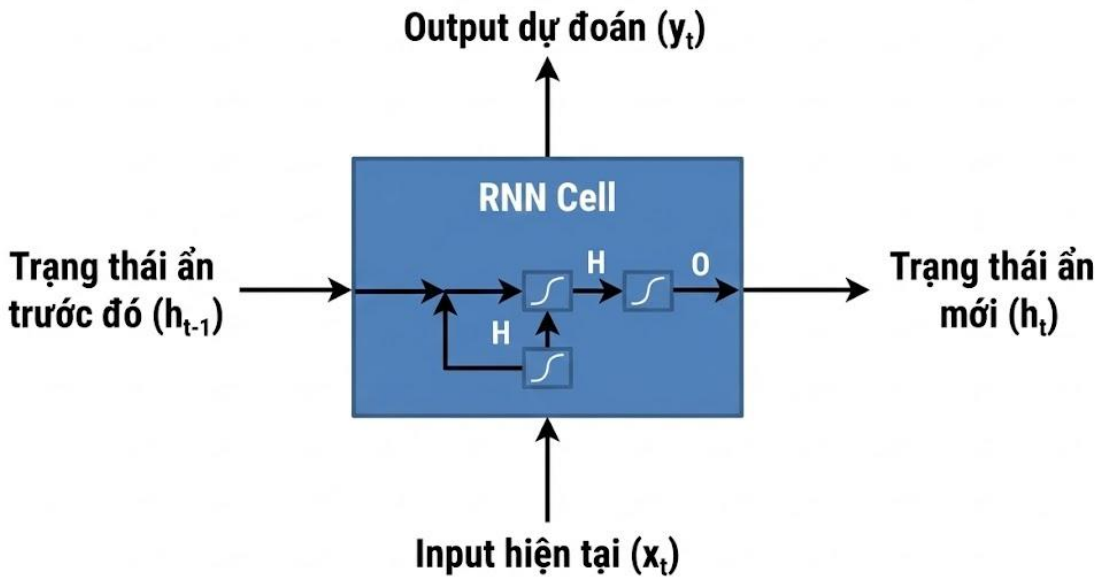
Many-to-One



Input: "Phim này chán quá" → Output: Tiêu cực (0).

KIẾN TRÚC TẾ BÀO RNN CƠ BẢN

- Tại mỗi bước thời gian t, RNN nhận 2 đầu vào:
  - **Input hiện tại (x_t):** Từ đang xét.
  - **Trạng thái ẩn trước đó (h_{t-1}):** Bộ nhớ từ quá khứ.
- Tạo ra 2 đầu ra:
  - **Output dự đoán (y_t):** (Tùy chọn).
  - **Trạng thái ẩn mới (h_t):** Chuyển sang bước $t + 1$.



CÔNG THỨC TOÁN HỌC (FORWARD PASS)

Tính trạng thái ẩn (Hidden State):

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

Tính đầu ra (Output):

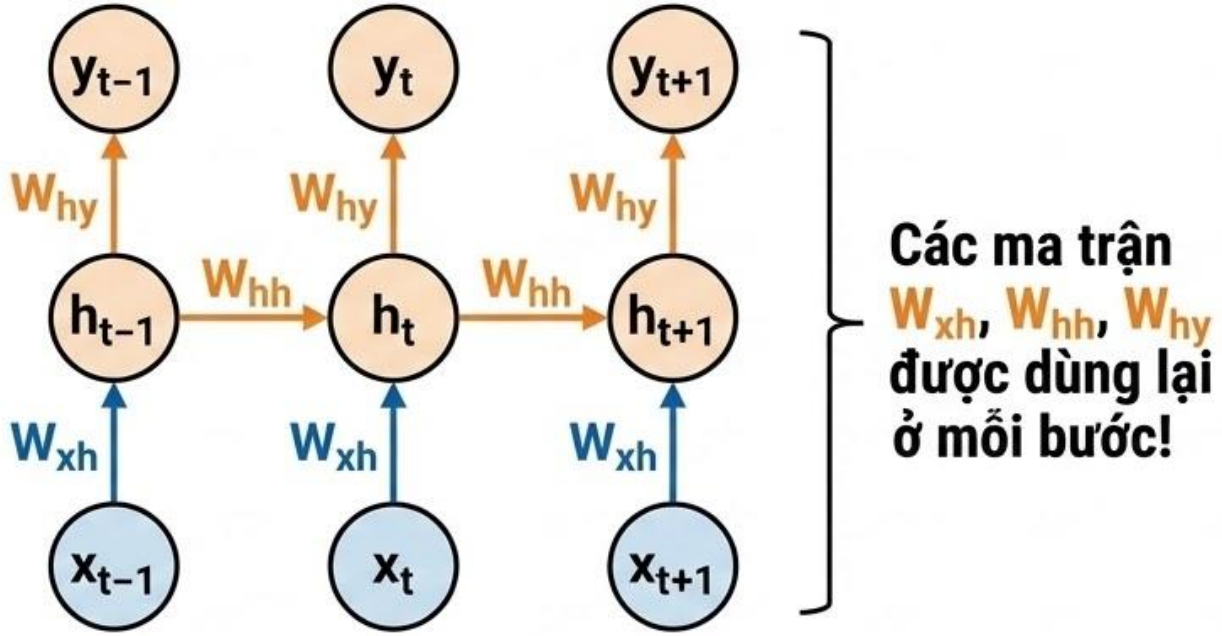
$$y_t = W_{hy}h_t + b_y$$

(Lưu ý: Hàm kích hoạt Tanh thường được dùng để giữ giá trị trong khoảng [-1, 1], tránh bùng nổ giá trị).

CHIA SẺ THAM SỐ (PARAMETER SHARING)

Đây là đặc điểm quan trọng nhất của RNN.

Các ma trận trọng số W_{xh}, W_{hh}, W_{hy} là giống nhau ở mọi bước thời gian t.



Minh họa: Unrolled RNN & Trọng số chia sẻ

Lợi ích:

Giảm số lượng tham số cần học.

Học một bộ trọng số thay vì một bộ cho mỗi bước thời gian.

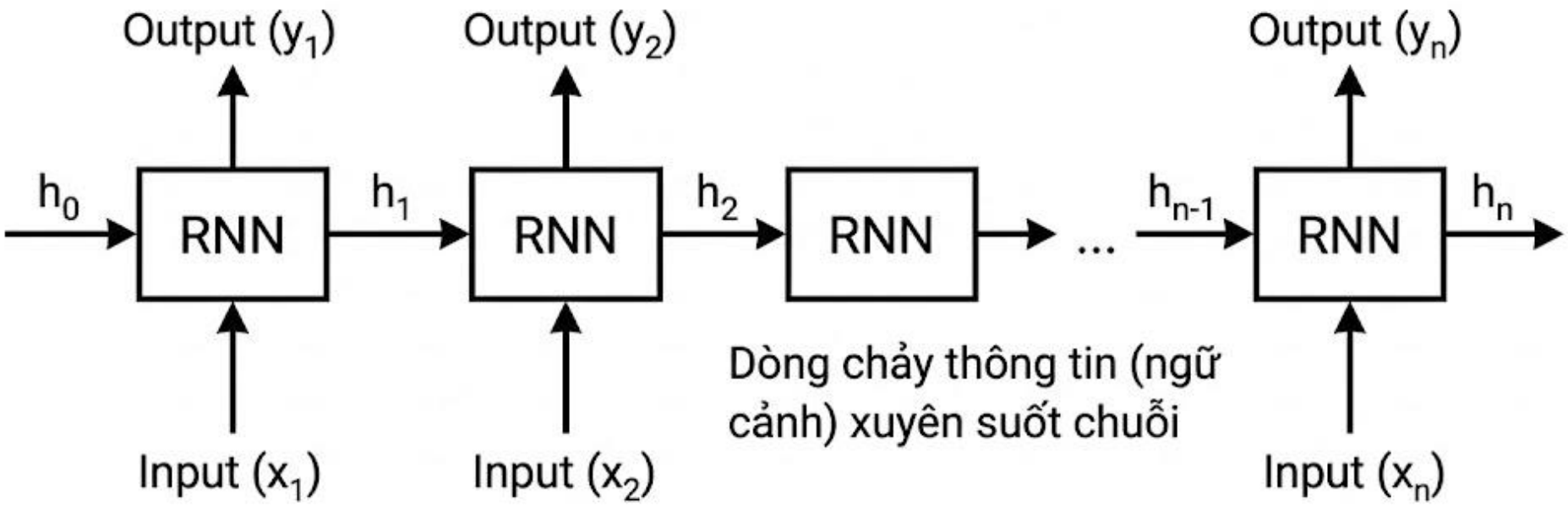
Cho phép mạng xử lý chuỗi có độ dài bất kỳ.

Mô hình linh hoạt với các câu ngắn, dài khác nhau.

TRỰC QUAN HÓA: UNROLLED RNN

$$x_1 \rightarrow [RNN] \rightarrow h_1 \rightarrow [RNN] \rightarrow h_2 \rightarrow \cdots$$

Mũi tên từ h_{t-1} sang h_t thể hiện dòng chảy thông tin (ngữ cảnh) xuyên suốt chuỗi.



KÍCH THƯỚC CÁC MA TRẬN

Giả sử: Input size D, Hidden size H.

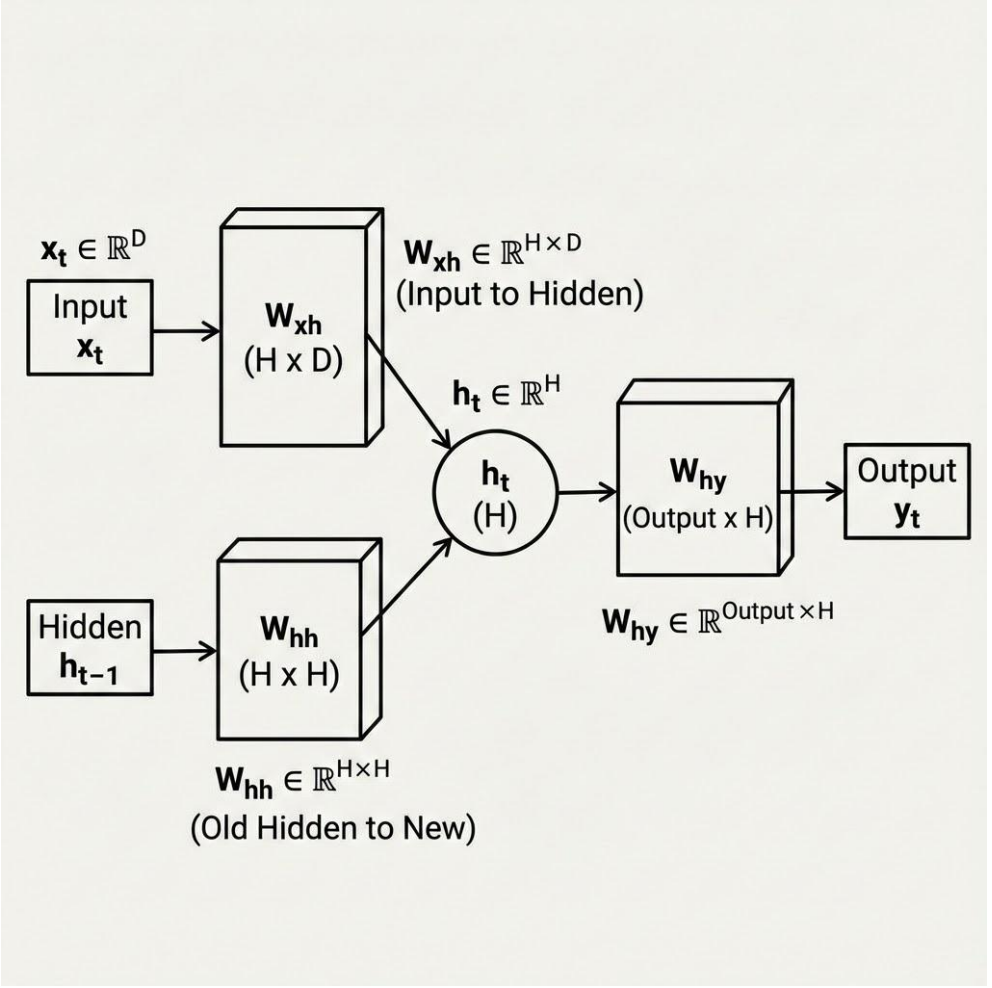
$x_t \in R^D$

$h_t \in R^H$

$W_{xh} \in R^{H \times D}$ (Chuyển input sang hidden)

$W_{hh} \in R^{H \times H}$ (Chuyển hidden cũ sang mới)

$W_{hy} \in R^{Output \times H}$



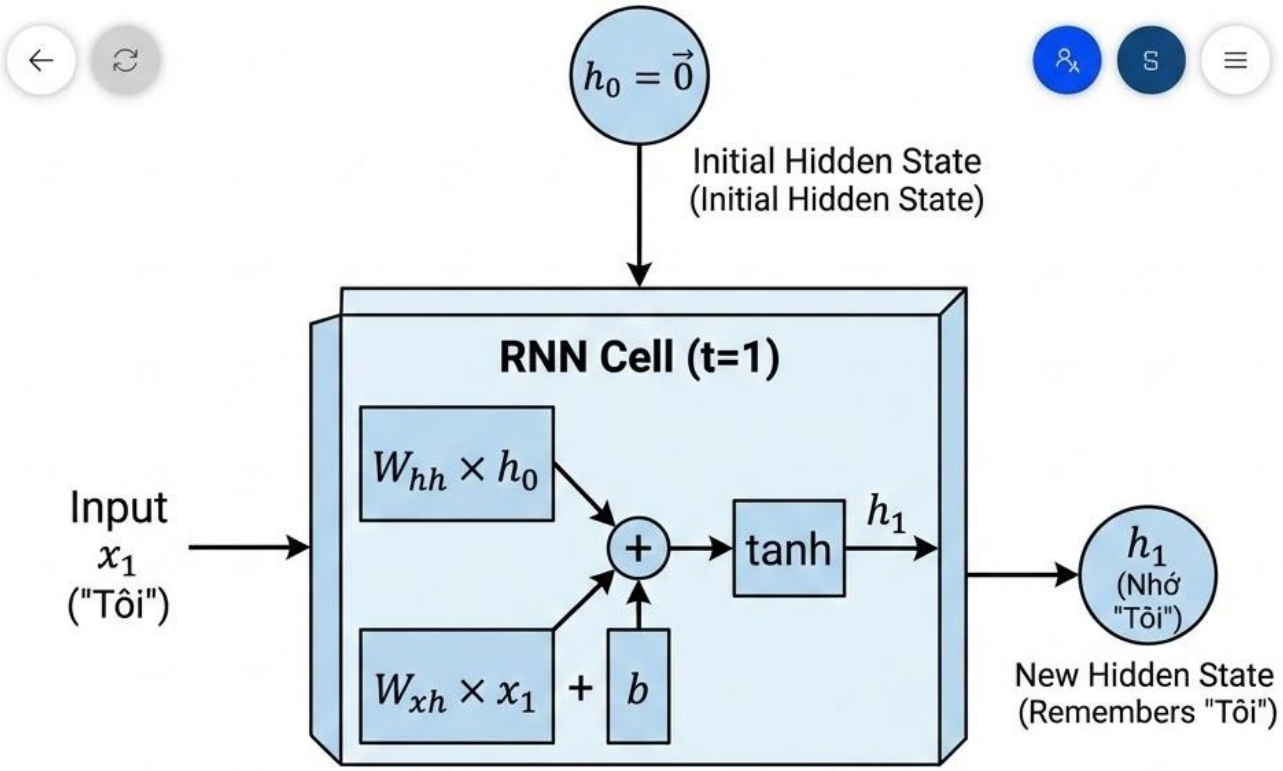
VÍ DỤ TÍNH TOÁN: BƯỚC 1

$h_0 = \vec{0}$ (Khởi tạo vector 0).

Input x_1 ("Tôi").

$h_1 = \tanh(W_{hh}h_0 + W_{xh}x_1 + b)$.

Mạng bắt đầu nhớ được từ "Tôi".



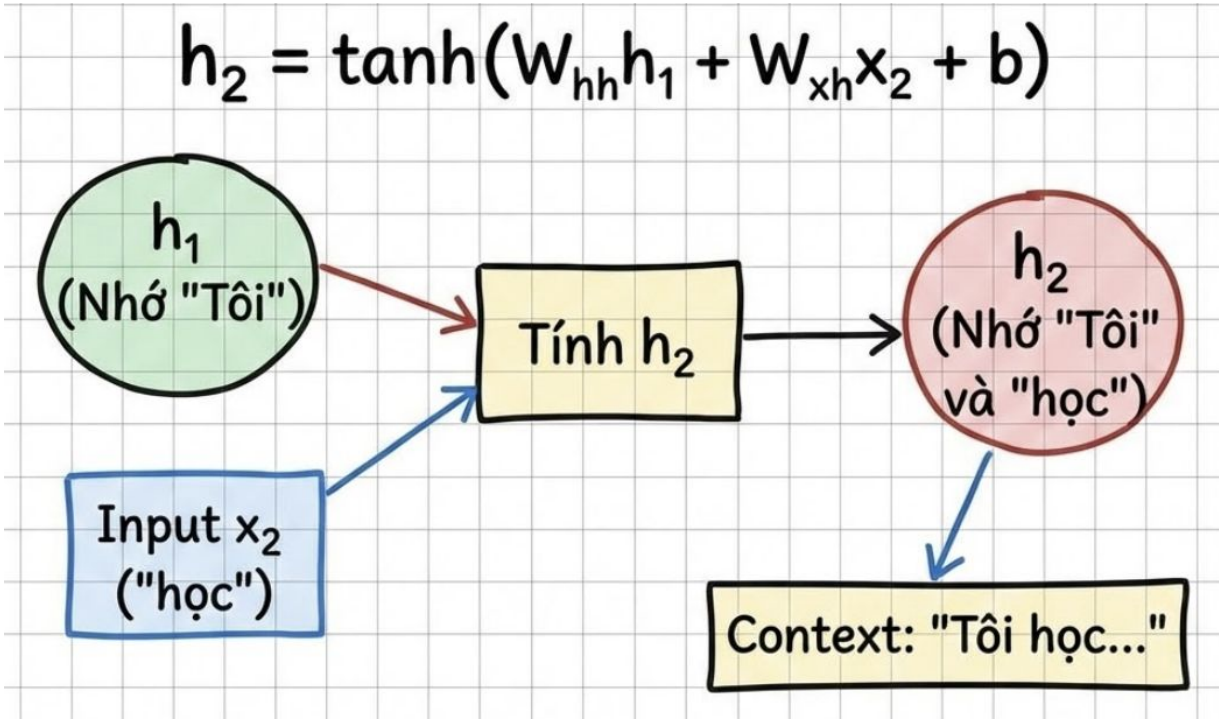
VÍ DỤ TÍNH TOÁN: BƯỚC 2

Input x_2 ("học").

$h_2 = \tanh(W_{hh}h_1 + W_{xh}x_2 + b)$.

h_2 chứa thông tin tổng hợp của cả "Tôi" và "học".

→ Context: "Tôi học...".



HÀM MẤT MÁT THEO THỜI GIAN

Với bài toán Many-to-Many (như dự đoán từ tiếp theo), Loss được tính tại mỗi bước t.

$L_t(\hat{y}_t, y_t)$: Ví dụ Cross-Entropy giữa dự đoán và thực tế tại bước t.

Tổng Loss của cả chuỗi:

$$L = \sum_{t=1}^{T} L_t$$

HẠN CHẾ CỦA RNN CƠ BẢN

Vấn đề

RNN cơ bản rất khó học các **phụ thuộc xa (Long-term dependencies)**.

Ví dụ

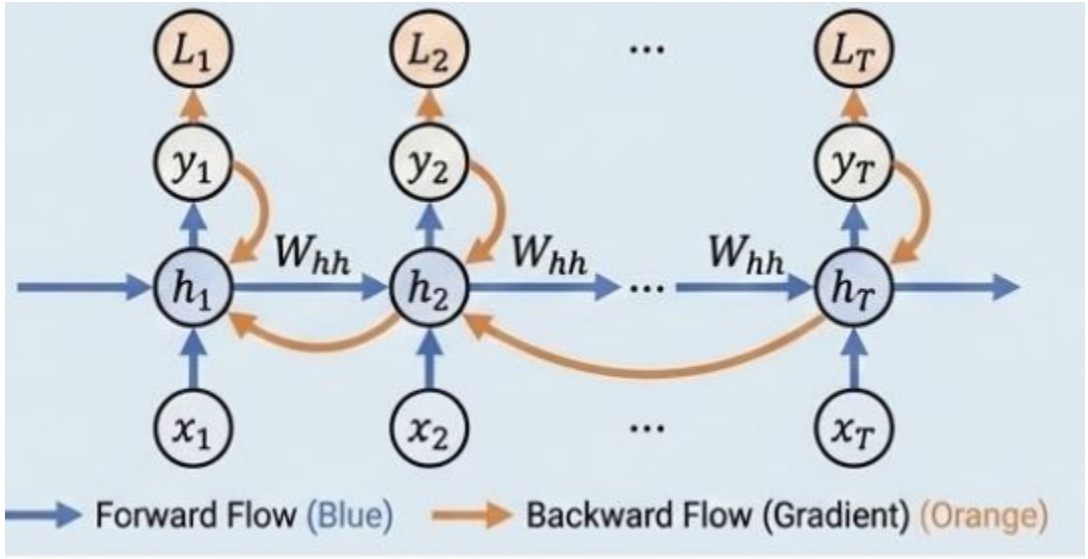
"Tôi sống ở Pháp... (100 từ)... tôi nói tiếng **Pháp**".

→ RNN thường quên từ "Pháp" ở đầu câu khi đi đến cuối câu do vấn đề biến mất đạo hàm.

BACKPROPAGATION THROUGH TIME (BPTT)

Để huấn luyện RNN, ta mở rộng mạng theo thời gian và áp dụng Lan truyền ngược.

Đạo hàm của Loss L theo W là tổng đạo hàm tại mỗi bước thời gian.



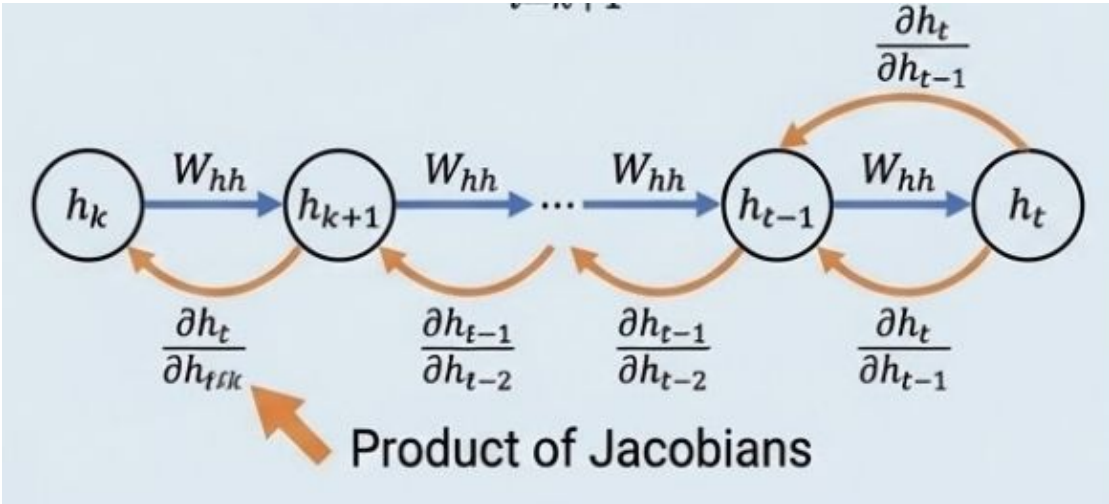
$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

QUY TẮC CHUỖI TRONG BPTT

Khi tính đạo hàm theo h_t, ta phải truy ngược về $h_{t-1}, h_{t-2}, \ldots, h_1$.

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}$$

Tích của rất nhiều ma trận Jacobians (W_{hh}) được nhân lại với nhau.

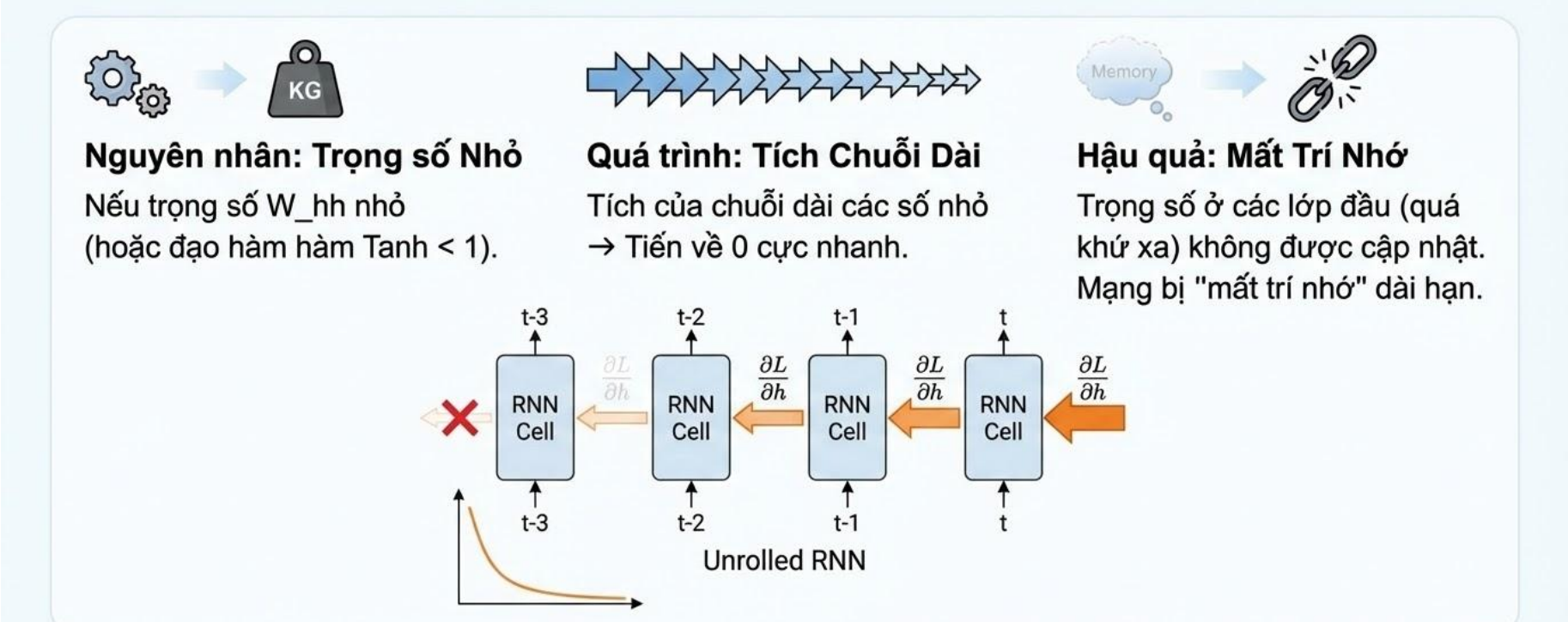


BIẾN MẤT ĐẠO HÀM (VANISHING GRADIENT)

Nếu trọng số W_{hh} nhỏ (hoặc đạo hàm hàm Tanh < 1).

Tích của chuỗi dài các số nhỏ → Tiến về 0 cực nhanh.

Hậu quả: Trọng số ở các lớp đầu (quá khứ xa) không được cập nhật. Mạng bị "mất trí nhớ" dài hạn.

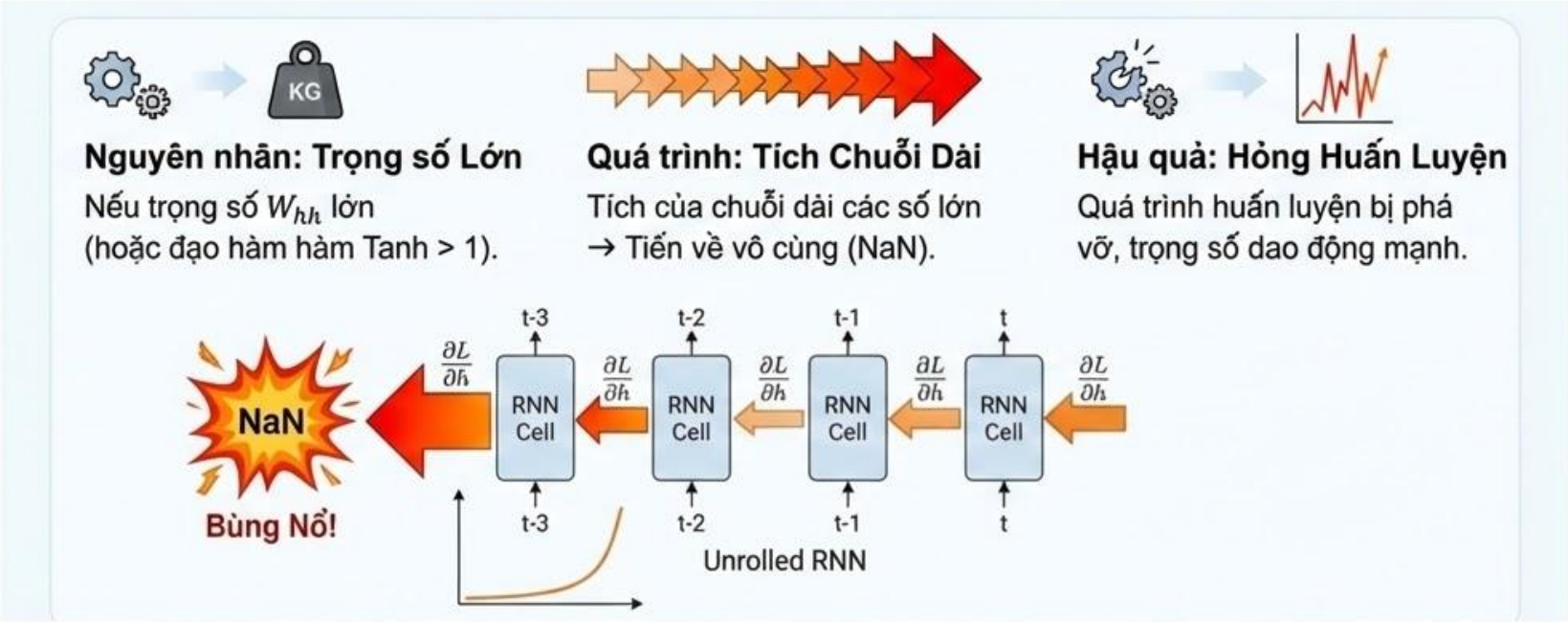


BÙNG NỔ ĐẠO HÀM (EXPLODING GRADIENT)

Ngược lại, nếu trọng số W_{hh} lớn.

Tích chuỗi dài → Tiến về vô cùng (NaN).

Hậu quả: Quá trình huấn luyện bị phá vỡ, trọng số dao động mạnh.

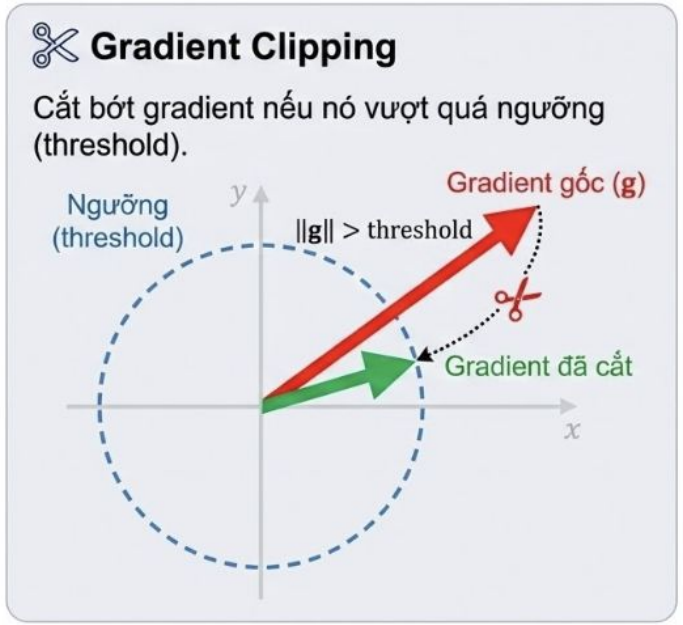


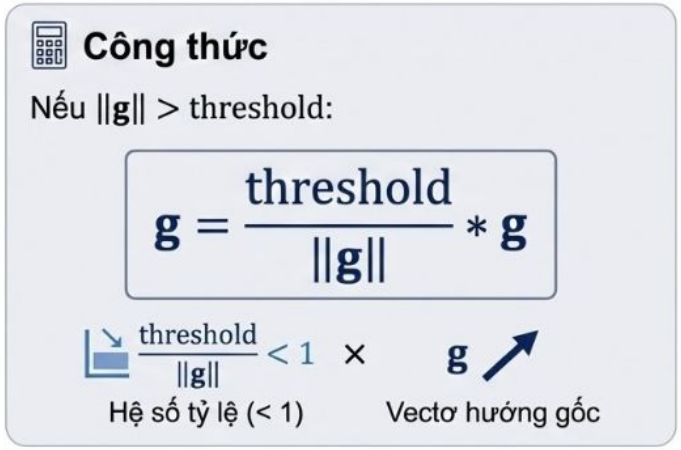
GIẢI PHÁP CHO EXPLODING GRADIENT

Gradient Clipping: Cắt bớt gradient nếu nó vượt quá ngưỡng (threshold).

Nếu $||g|| > threshold$, gán $g = \frac{threshold}{||g||} g$.

Giữ hướng đi nhưng giảm độ lớn bước nhảy.





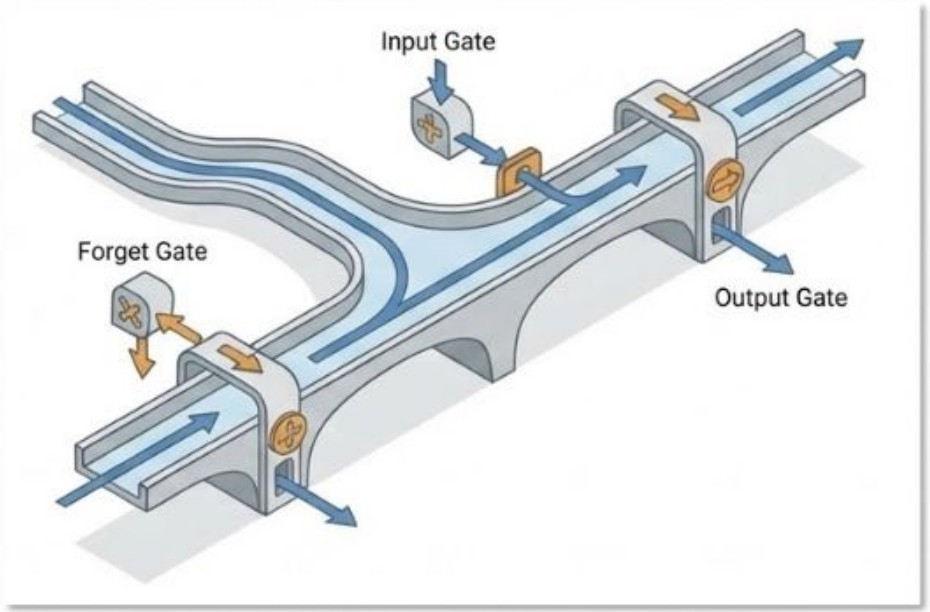


GIẢI PHÁP CHO VANISHING GRADIENT

- Thay đổi hàm kích hoạt (Sử dụng **ReLU**).
- Khởi tạo trọng số thông minh.
- **Giải pháp tốt nhất:** Sử dụng kiến trúc phức tạp hơn như **LSTM** hoặc **GRU**.

LONG SHORT-TERM MEMORY (LSTM)

- Được giới thiệu bởi **Hochreiter & Schmidhuber (1997)**.
- Thiết kế đặc biệt để tránh Vanishing Gradient.
- Có khả năng học phụ thuộc xa (Long-term) tốt hơn nhiều so với RNN chuẩn.



Cơ chế 'Cổng' và 'Cell State' cho phép học phụ thuộc xa.

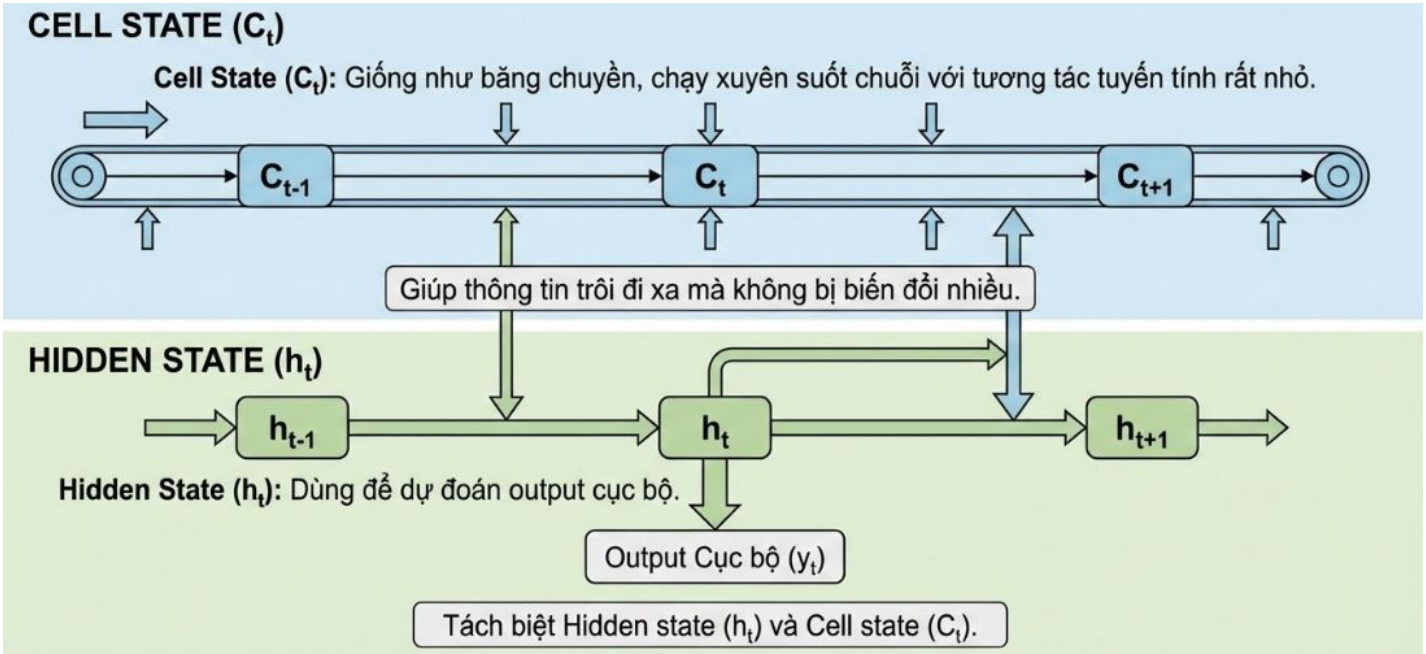
Ý TƯỞNG CỐT LÕI: CELL STATE (C_t)

Tách biệt **Hidden state (h_t)** và **Cell state (C_t)**.

Cell State (C_t): Giống như băng chuyền, chạy xuyên suốt chuỗi với tương tác tuyến tính rất nhỏ.

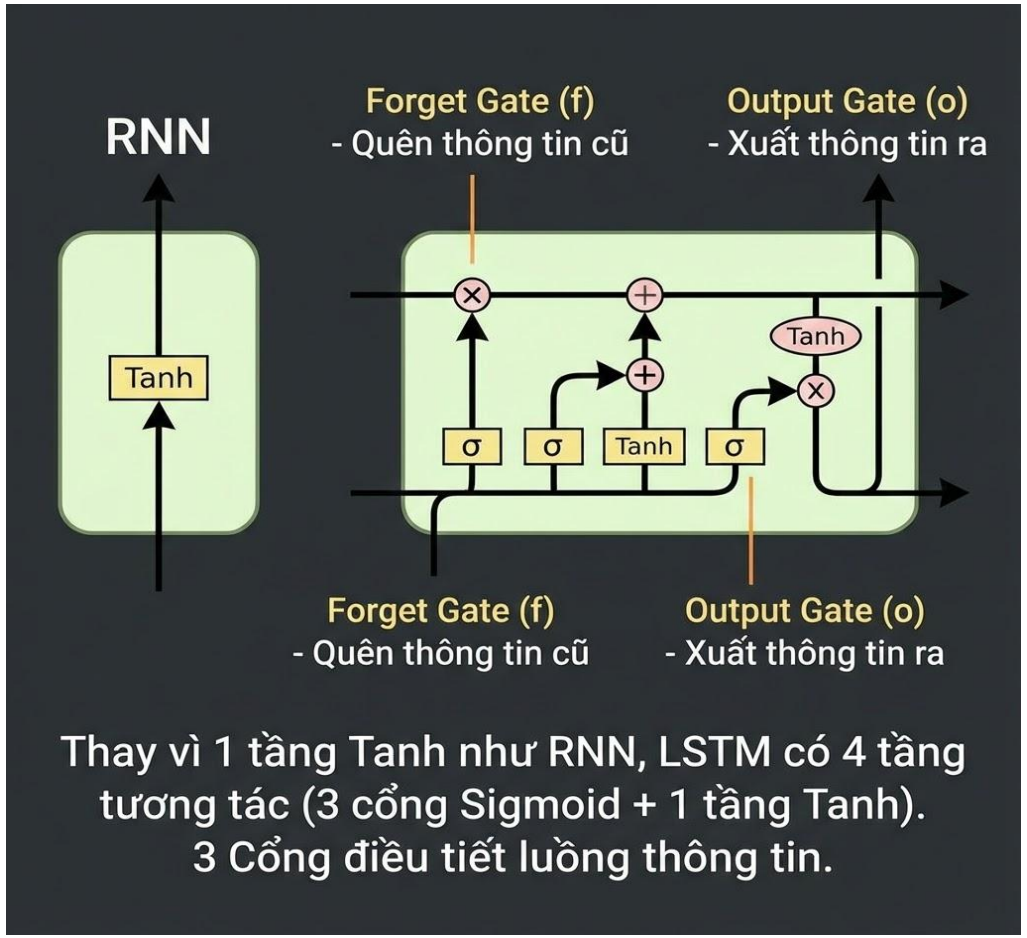
Giúp thông tin trôi đi xa mà không bị biến đổi nhiều.

Hidden State (h_t): Dùng để dự đoán output cục bộ.



CẤU TRÚC BÊN TRONG LSTM

- Thay vì 1 tầng Tanh như RNN, LSTM có 4 tầng tương tác (3 cổng Sigmoid + 1 tầng Tanh).
- 3 Cổng (Gates) điều tiết luồng thông tin:
  1. **Forget Gate (f):** Quên thông tin cũ.
  2. **Input Gate (i):** Nạp thông tin mới.
  3. **Output Gate (o):** Xuất thông tin ra.



BƯỚC 1: FORGET GATE (CỔNG QUÊN)

Quyết định bỏ bao nhiêu % thông tin từ Cell state cũ (C_{t-1}).

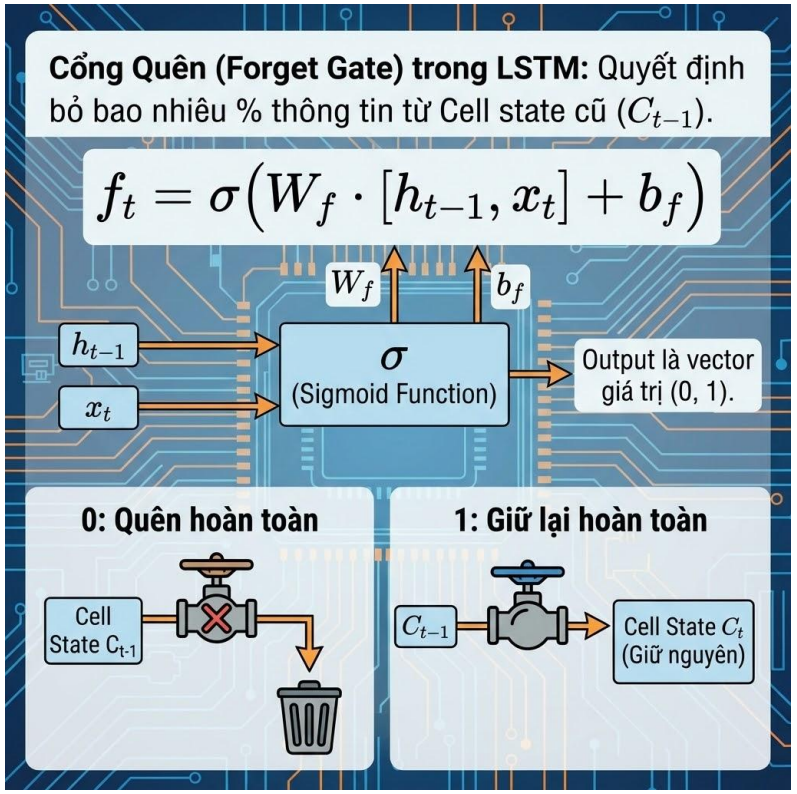
Công thức:

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

Output là vector giá trị $(0,1)$.

0: Quên hoàn toàn.

1: Giữ lại hoàn toàn.



BƯỚC 2: INPUT GATE (CỔNG VÀO)

Quyết định cập nhật thông tin gì vào Cell state.

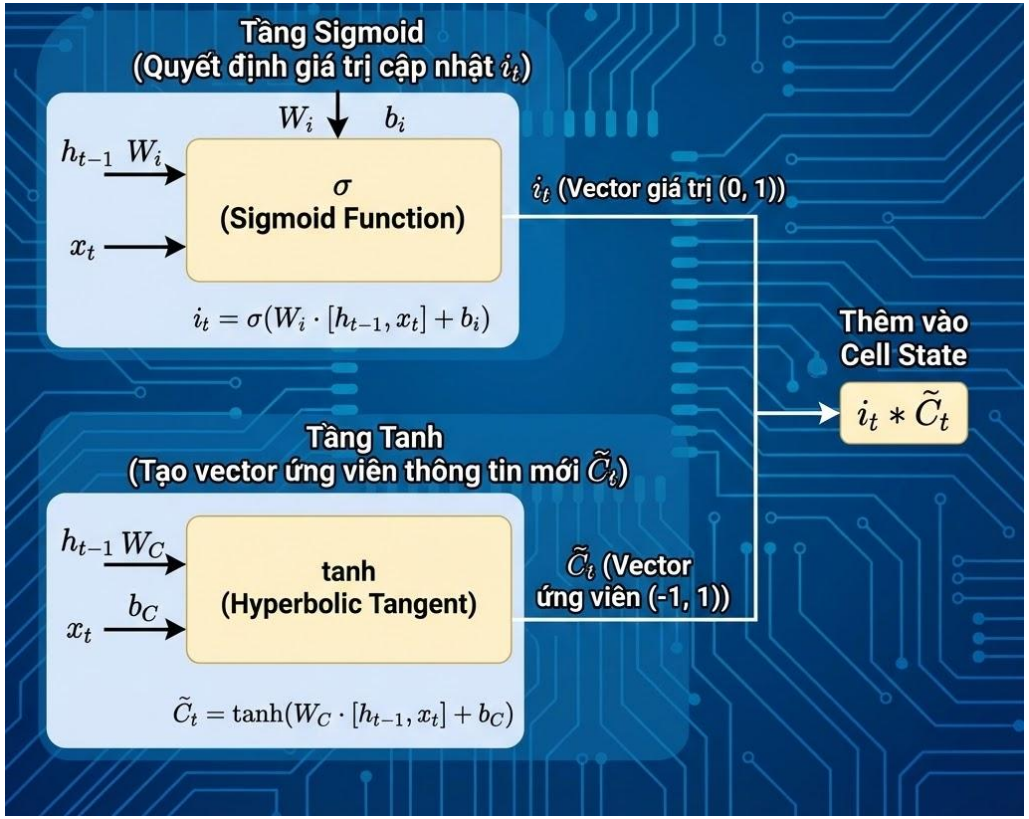
Gồm 2 phần:

Tầng Sigmoid quyết định cập nhật giá trị nào (i_t):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Tầng Tanh tạo ra vector ứng viên thông tin mới ($\widetilde{C}_t$):

$$\widetilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



BƯỚC 3: CẬP NHẬT TẾ BÀO (UPDATE)

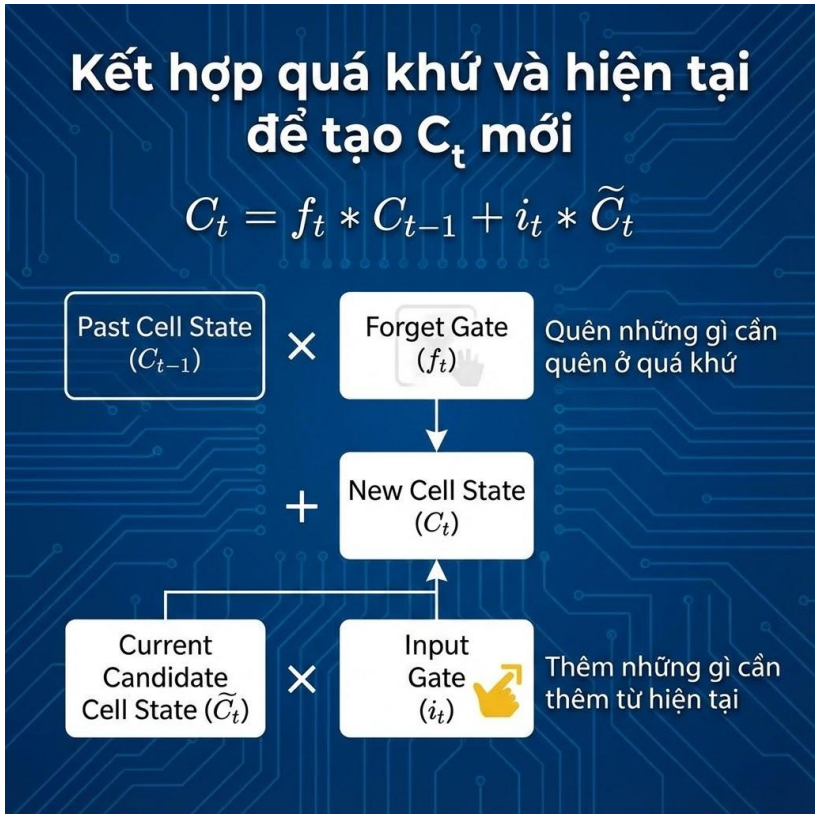
Kết hợp quá khứ và hiện tại để tạo C_t mới.

$$C_t = f_t * C_{t-1} + i_t * \widetilde{C}_t$$

Giải thích:

$f_t * C_{t-1}$: Quên những gì cần quên ở quá khứ.

$i_t * \widetilde{C}_t$: Thêm những gì cần thêm từ hiện tại.



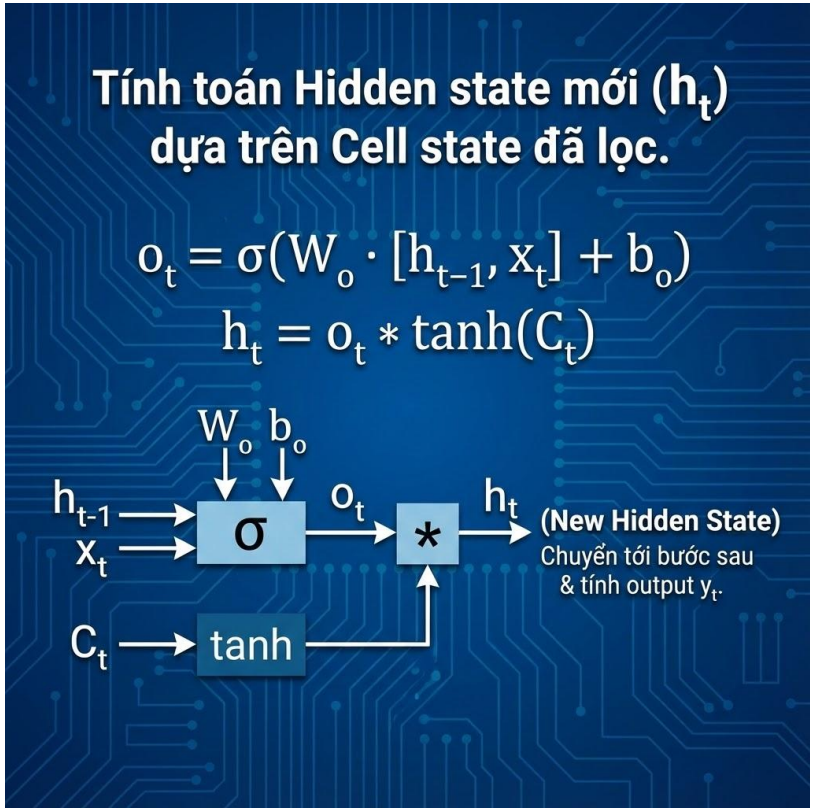
BƯỚC 4: OUTPUT GATE (CỔNG RA)

Tính toán Hidden state mới (h_t) dựa trên Cell state đã lọc.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

h_t này sẽ được chuyển tới bước sau và dùng để tính output y_t.



TẠI SAO LSTM TRÁNH ĐƯỢC VANISHING?

Nhờ công thức cộng trong cập nhật C_t:

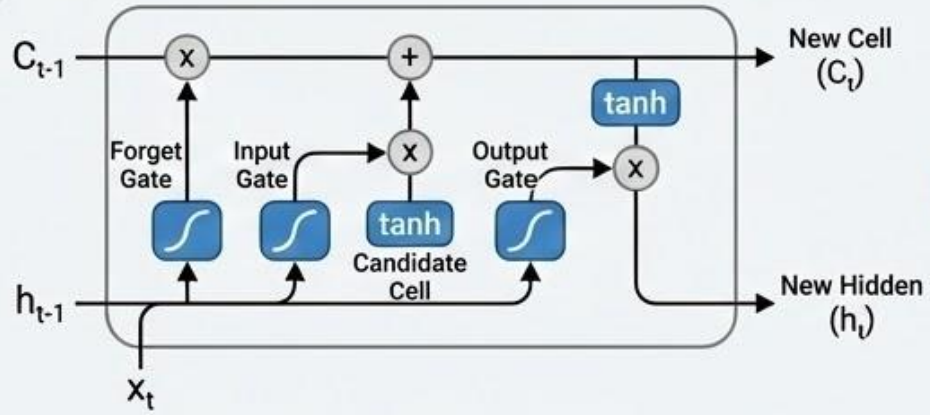
$$C_t = f_t * C_{t-1} + \cdots$$

Đạo hàm truyền ngược qua phép cộng (+) không bị suy giảm nhanh như phép nhân ma trận trong RNN thường.

Cổng Forget cho phép Gradient truyền nguyên vẹn qua nhiều bước thời gian.

GRU (GATED RECURRENT UNIT)

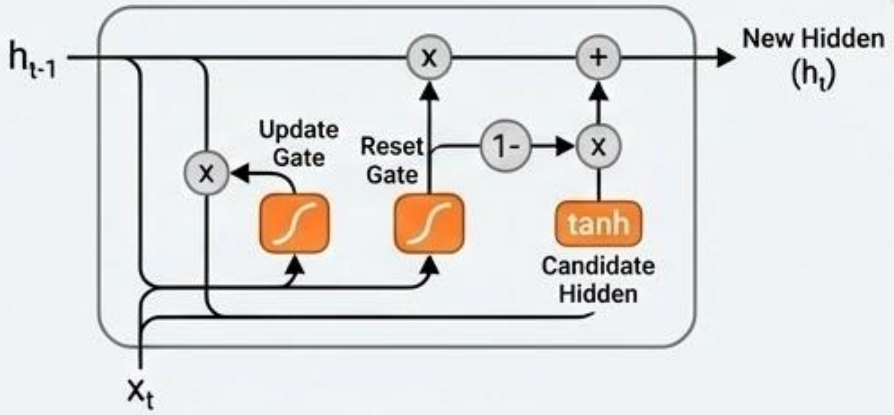
LSTM (1997): Cấu trúc Phức tạp



- 3 Cổng (Forget, Input, Output)
- Trạng thái Cell (C_t) và Hidden (h_t) riêng biệt

Đơn giản hóa

GRU (2014): Cấu trúc Đơn giản hóa



- 2 Cổng (Update, Reset)
- Gộp Cell (C_t) và Hidden (h_t) thành một

Đặc điểm Nổi bật

Gộp Trạng thái
C_t và h_t được hợp nhất, giảm độ phức tạp và bộ nhớ.

Hai Cổng Hiệu quả
Chỉ sử dụng Update Gate và Reset Gate để kiểm soát luồng thông tin.

Tính toán Nhanh hơn
Ít tham số hơn dẫn đến thời gian huấn luyện và suy luận nhanh hơn.

Hiệu suất Tương đương
Đạt được kết quả tương tự LSTM trên nhiều tác vụ tuần tự.

SO SÁNH RNN VS LSTM

Đặc điểm	RNN	LSTM
Cấu trúc	Đơn giản (1 tầng Tanh)	Phức tạp (4 tầng, 3 cổng)
Bộ nhớ	Ngắn hạn	Dài hạn (Long Short-term)
Huấn luyện	Dễ bị Vanishing Grad	Ổn định hơn với chuỗi dài
Tốc độ	Nhanh	Chậm hơn (nhiều tham số)

ỨNG DỤNG 1: MÔ TẢ HÌNH ẢNH

Mục tiêu: Input ảnh → Output câu mô tả "Người đàn ông đang cưỡi ngựa".

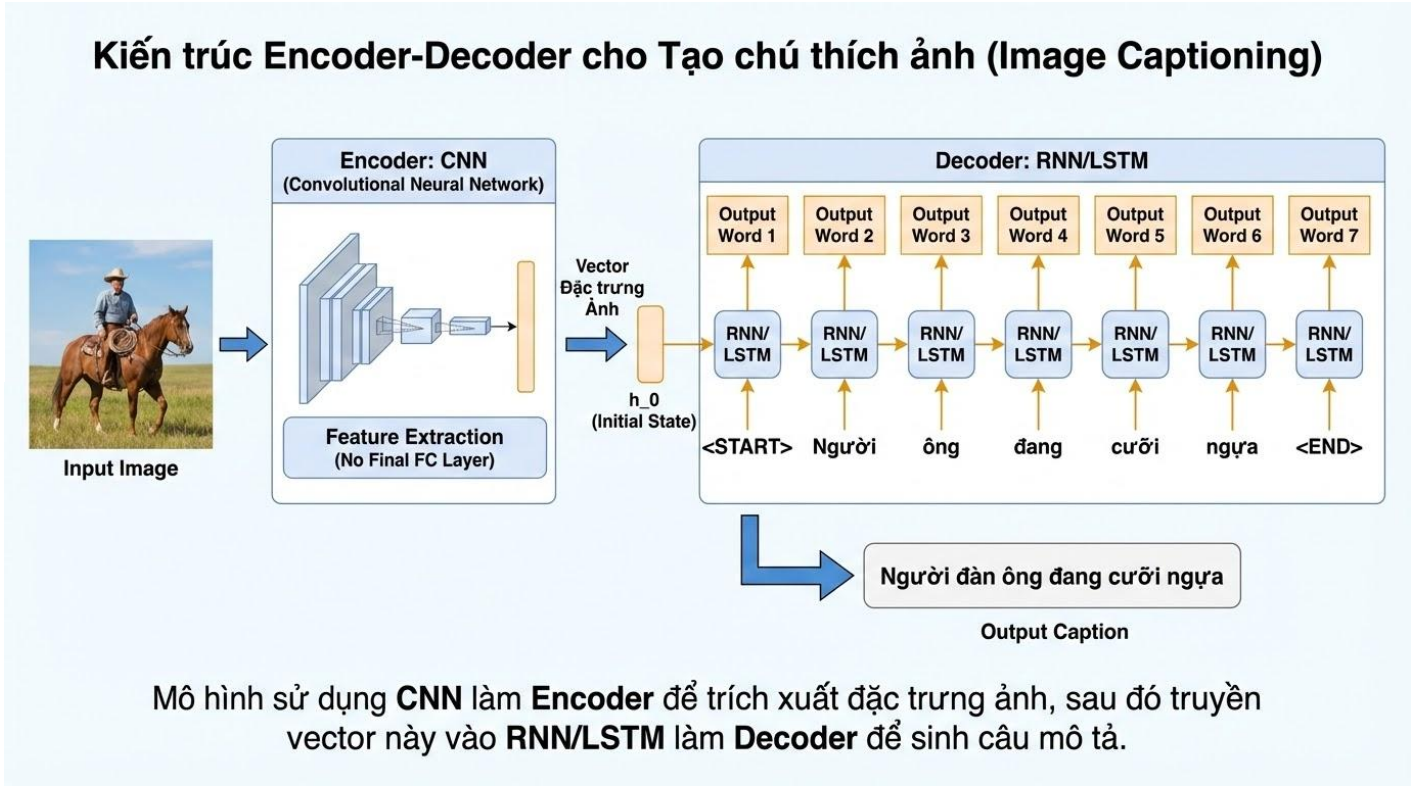
Kiến trúc: Encoder-Decoder.

Encoder: CNN (Bỏ lớp FC cuối) để trích xuất vector đặc trưng ảnh.

Decoder: RNN/LSTM nhận vector ảnh làm trạng thái khởi tạo (h_0) hoặc input đầu tiên.

QUY TRÌNH IMAGE CAPTIONING

1. Đưa ảnh qua CNN (VGG/ResNet) → Vector đặc trưng V.
2. Khởi tạo LSTM với $h_0 = Linear(V)$.
3. Bước 1: Input <START>, LSTM dự đoán từ "Người".
4. Bước 2: Input "Người", LSTM dự đoán "đàn ông".
5. ... Lặp lại đến khi gặp từ <END>.



ỨNG DỤNG 2: PHÂN TÍCH CẢM XÚC

Bài toán

Phân loại bình luận là Tích cực hay Tiêu cực.

Input

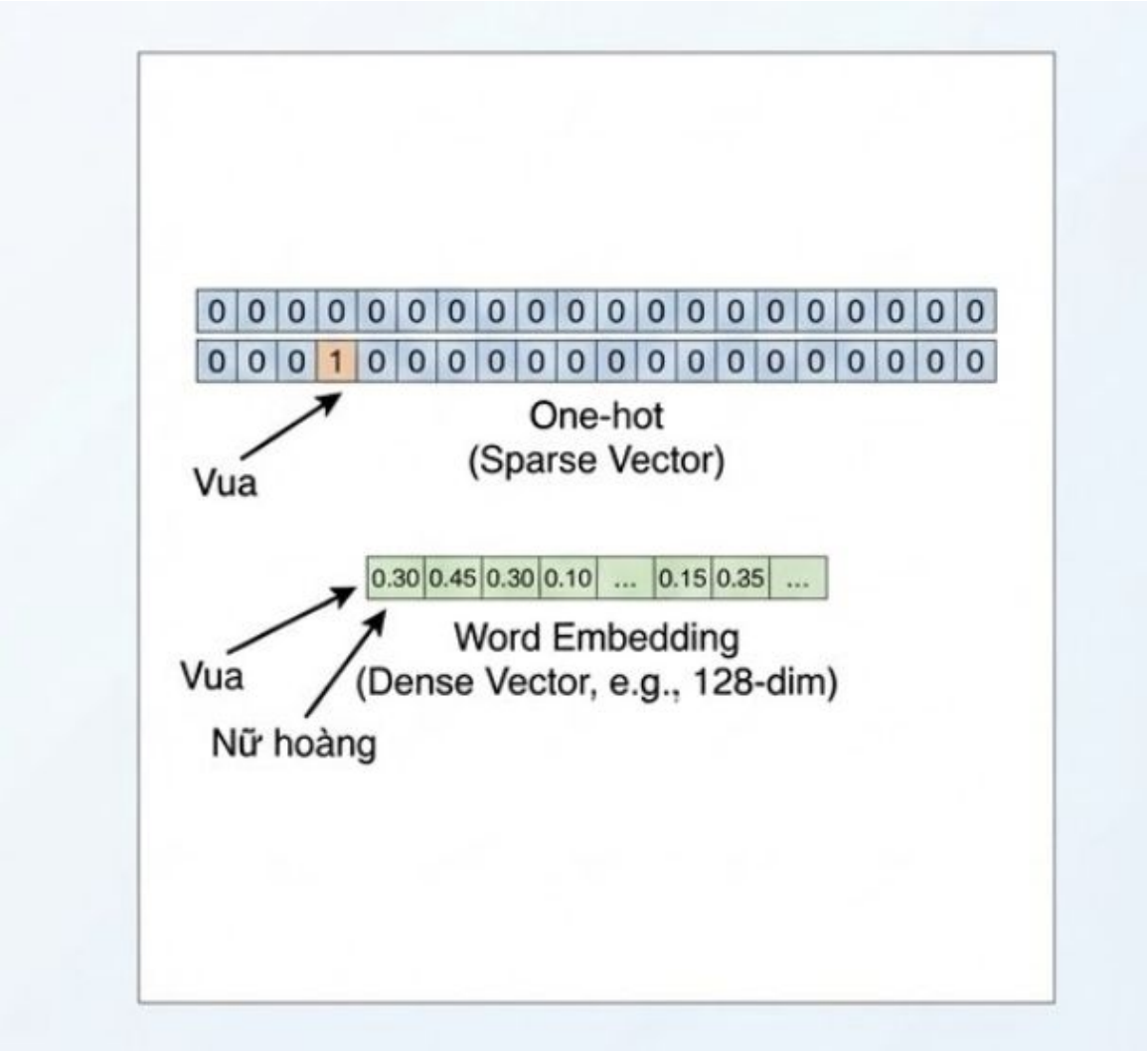
Chuỗi văn bản có độ dài thay đổi.

Mô hình

Many-to-One LSTM.

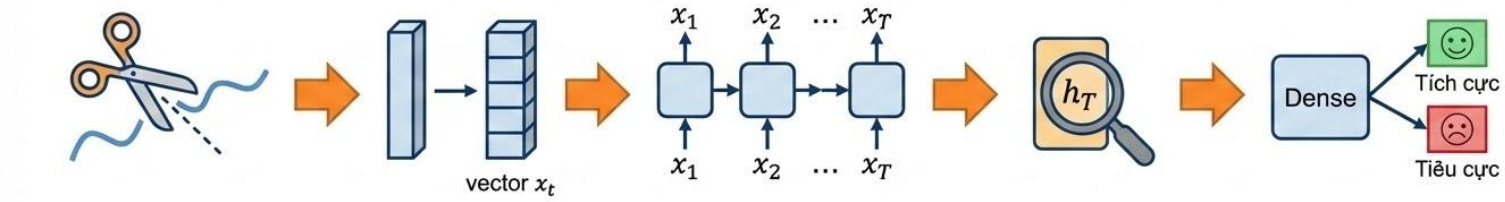
EMBEDDING LAYER (BIỂU DIỄN TỪ)

- Máy tính không hiểu chữ. Cần chuyển từ thành số.
- **One-hot:** Vector quá dài và thưa (Sparse).
- **Word Embedding (Word2Vec/GloVe):** Chuyển mỗi từ thành vector đặc (Dense) kích thước cố định, thể hiện được ngữ nghĩa tương đồng.



QUY TRÌNH PHÂN TÍCH CẢM XÚC

1. **Tokenization:** Tách câu thành các từ.
2. **Embedding:** Chuyển từ thành vector x_t.
3. **LSTM Layer:** Đưa lần lượt các vector vào LSTM.
4. **Feature Extraction:** Lấy trạng thái ẩn cuối cùng h_T (chứa thông tin tóm tắt cả câu).
5. **Classification:** Đưa h_T qua lớp Dense + Sigmoid/Softmax để phân loại.



1. Tokenization	2. Embedding	3. LSTM Layer	4. Feature Extraction	5. Classification
Tách câu thành các từ.	Chuyển từ thành vector x_t.	Đưa lần lượt các vector vào LSTM.	Lấy trạng thái ẩn cuối cùng h_T (chứa thông tin tóm tắt cả câu).	Đưa h_T qua lớp Dense + Sigmoid/Softmax để phân loại.

CODE MINH HỌA KERAS

```
model = Sequential()
model.add(Embedding(input_dim=10000, output_dim=128))
model.add(LSTM(64, return_sequences=False))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam')
```

TỔNG KẾT BÀI HỌC

- **RNN:** Mạng có hồi quy, chuyên trị dữ liệu tuần tự, chia sẻ trọng số.
- **Vấn đề:** BPTT gây ra Vanishing Gradient trên chuỗi dài.
- **LSTM:** Giải pháp với Cell State và cơ chế Cổng (Forget, Input, Output).
- **Ứng dụng:** Rất mạnh trong NLP (Dịch máy, Chatbot) và Vision (Captioning).

CÂU HỎI & BÀI TẬP

1. Tại sao nói C_t trong LSTM giống như "đường cao tốc"?
2. Vẽ sơ đồ kiến trúc Many-to-Many cho bài toán Dịch máy (Encoder-Decoder).
3. **Bài tập:** Sử dụng bộ dữ liệu IMDB trong Keras để huấn luyện mô hình LSTM phân loại review phim.